Task-04 Password Security and Authentication Analysis

Chapter 1: Introduction to Password Security

Passwords are one of the most used methods of authentication in computer systems. They are widely used across operating systems, applications, and online services to verify user identity and control access to sensitive information. Because of this, password security plays a crucial role in protecting systems from unauthorized access.

Despite their importance, passwords are often targeted by attackers due to weak user choices and improper storage practices. Storing passwords in plain text or using outdated security methods can lead to serious security breaches. To prevent this, modern systems avoid storing passwords directly and instead use cryptographic techniques to protect them.

The most common approach is password hashing, where a password is converted into a fixed-length hash value using a hashing algorithm. During authentication, the entered password is hashed again and compared with the stored hash, ensuring that the actual password is never stored or exposed.

This chapter introduces the basic concepts of password security and explains why secure password storage is essential. It also lays the foundation for understanding password attacks and hash-cracking techniques demonstrated in later chapters of this task.

Chapter 2: Hashing vs Encryption

Hashing and encryption are both cryptographic techniques used to protect data, but they serve different purposes and work in different ways. Understanding this difference is essential for password security.

Encryption is a reversible process. It converts data into an unreadable format using an encryption algorithm and a key. The original data can be recovered by decrypting it with the correct key. Encryption is mainly used to protect data that needs to be read again later, such as files, messages, or database records.

Hashing, on the other hand, is a one-way process. When a password is hashed, it is transformed into a fixed-length string called a hash value. This process cannot be reversed to obtain the original password. Every time the same password is hashed using the same algorithm, it produces the same hash output.

Because hashing is irreversible, it is ideal for password storage. When a user logs in, the system hashes the entered password and compares it with the stored hash. If both hashes match, access is granted—without ever storing or revealing the actual password.

In password security, hashing is preferred over encryption because passwords do not need to be recovered, only verified. If encrypted passwords were stolen, attackers could decrypt them using the key. However, if hashed passwords are leaked, attackers must attempt to guess the original password through attacks such as dictionary or brute-force attacks.

Key Differences Between Hashing and Encryption:

Feature	**Hashing**	**Encryption**
Reversible	No	Yes
Uses a key	No	Yes
Purpose	Password verification	Data protection
Common use	Password storage	Files, messages, databases

Chapter 3: Common Hashing Algorithms

Different hashing algorithms are used to store passwords, but not all of them provide the same level of security. Over time, some algorithms have become insecure due to advances in computing power and attack techniques. This chapter discusses commonly used hashing algorithms and explains their strengths and weaknesses in the context of password security.

3.1 MD5 (Message Digest Algorithm 5)

MD5 is one of the earliest and most well-known hashing algorithms. It produces a 128-bit hash value, typically represented as a 32-character hexadecimal string. MD5 was originally designed to ensure data integrity, not for secure password storage.

MD5 is considered insecure today because it is extremely fast. Attackers can generate and compare millions of MD5 hashes per second, making it highly vulnerable to dictionary and brute-force attacks. As demonstrated in the practical part of this task, weak passwords hashed using MD5 can be cracked almost instantly.

Due to these weaknesses, MD5 is no longer recommended for password storage in modern systems.

3.2 SHA-1 and SHA-256 (Secure Hash Algorithms)

The Secure Hash Algorithm (SHA) family was developed to improve upon earlier hashing techniques like MD5.

SHA-1 produces a 160-bit hash value and was once widely used for security purposes. However, it has been proven vulnerable to cryptographic attacks and is now considered insecure for password protection.

SHA-256, part of the SHA-2 family, produces a 256-bit hash and is more secure than both MD5 and SHA-1. While SHA-256 provides stronger resistance against collisions, it is still relatively fast. This speed makes it vulnerable to dictionary attacks when weak passwords are used, as shown during the practical experiments.

Although SHA-256 is suitable for data integrity and digital signatures, it is not ideal on its own for password storage.

3.3 bcrypt

bcrypt is a password hashing algorithm specifically designed for secure password storage. Unlike MD5 and SHA algorithms, bcrypt is intentionally slow and computationally expensive. It includes a built-in salt and allows the use of a configurable cost factor, which determines how much processing power is required to compute each hash.

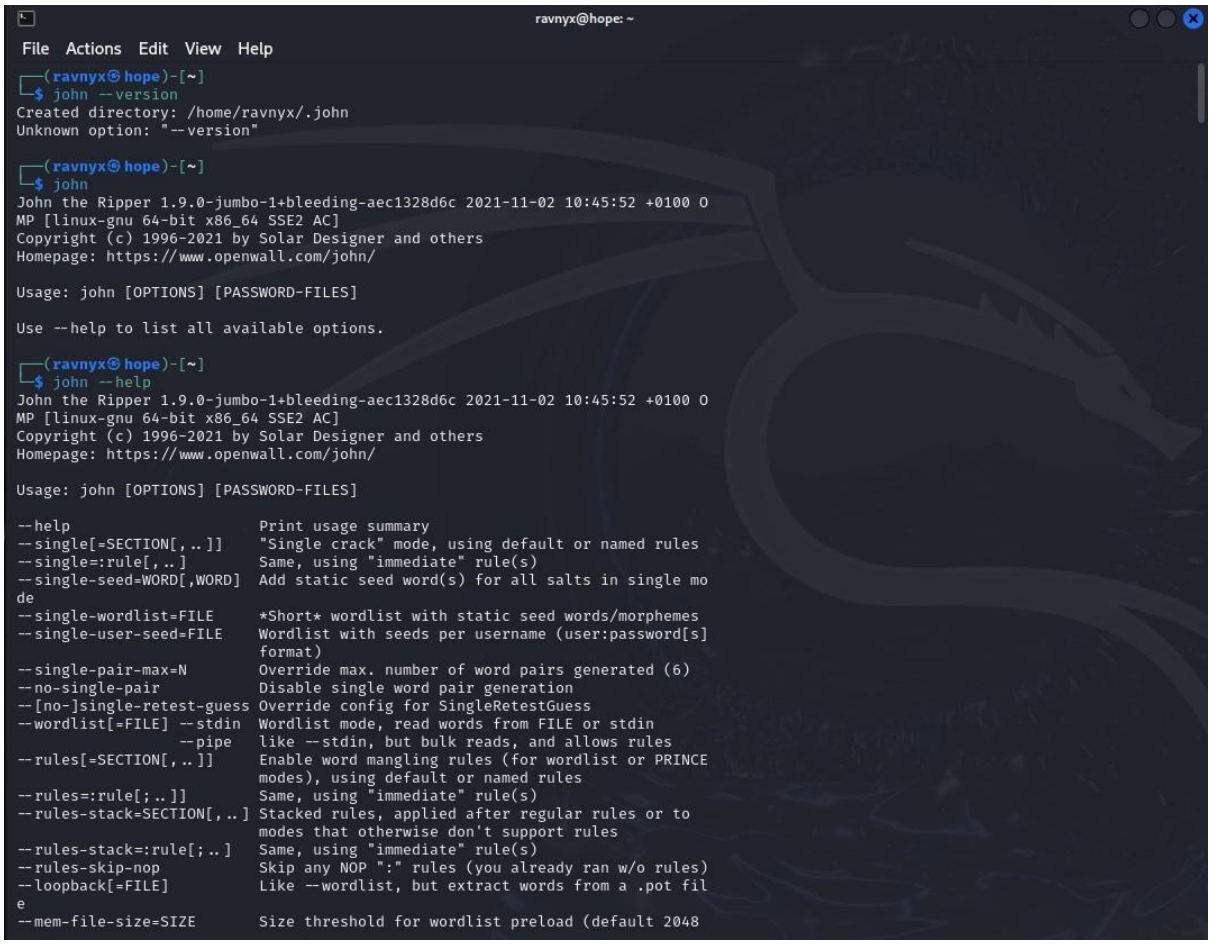
The slowness of bcrypt makes large-scale password cracking attacks significantly more difficult. Even if attackers obtain the hashed passwords, attempting to crack them becomes time-consuming and resource-intensive.

In the practical section of this task, bcrypt hashes required noticeably more time to crack compared to MD5 and SHA-256. This demonstrates why bcrypt is widely recommended for storing passwords in modern authentication systems.

3.4 Comparison of Hashing Algorithms

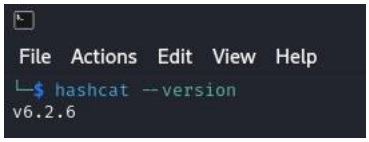
Algorithm	**Speed**	**Designed for Passwords**	**Security Status**
MD5	Very fast	No	Insecure
SHA-1	Fast	No	Insecure
SHA-256	Fast	No	Moderately secure
bcrypt	Slow	Yes	Secure

Figure3.1: John the Ripper Tool Verification



Note: This screenshot confirms that John the Ripper is installed and accessible in the Kali Linux environment. The tool is verified before performing password hash cracking experiments.

Figure 3.2: Hashcat Tool Verification



Note: This screenshot verifies the availability of Hashcat, a high-performance password recovery tool used for cracking MD5 and SHA-256 hashes during the practical analysis.

(All practical experiments were performed on Kali Linux using John the Ripper and Hashcat with the rockyou.txt wordlist.)

Chapter 4: Password Attacks

Password attacks are techniques used by attackers to recover plaintext passwords from stored password hashes. These attacks do not break the hashing algorithm itself; instead, they exploit weak passwords, poor password policies, and fast hashing mechanisms. Understanding these attack methods is essential for analyzing why certain passwords fail and how systems can be better protected.

4.1 Dictionary Attacks

A dictionary attack is one of the most common password-cracking techniques. In this method, attackers use a predefined list of commonly used passwords, known as a **wordlist**, and attempt to match their hash values with the target hash.

These wordlists often contain:

- Common passwords
- Leaked passwords from real data breaches
- Variations of frequently used words

In the practical part of this task, the rockyou.txt wordlist was used. This wordlist contains millions of real-world passwords and is highly effective against weak and commonly used passwords such as “password123” and “letmein”.

Dictionary attacks are fast and efficient when users choose predictable passwords. If a password exists in the wordlist, it can often be cracked within seconds.

4.2 Brute-Force Attacks

A brute-force attack is a more exhaustive technique in which attackers try every possible combination of characters until the correct password is found. Unlike dictionary attacks, brute-force attacks do not rely on predefined wordlists.

Brute-force attacks are:

- Guaranteed to succeed eventually
- Extremely time-consuming

- Highly dependent on password length and complexity

For example, a short password using only lowercase letters can be cracked relatively quickly, whereas a long password with mixed characters may take years to crack. Modern systems often mitigate brute-force attacks by implementing account lockouts, rate limiting, and CAPTCHA mechanisms.

4.3 Dictionary Attack vs Brute-Force Attack

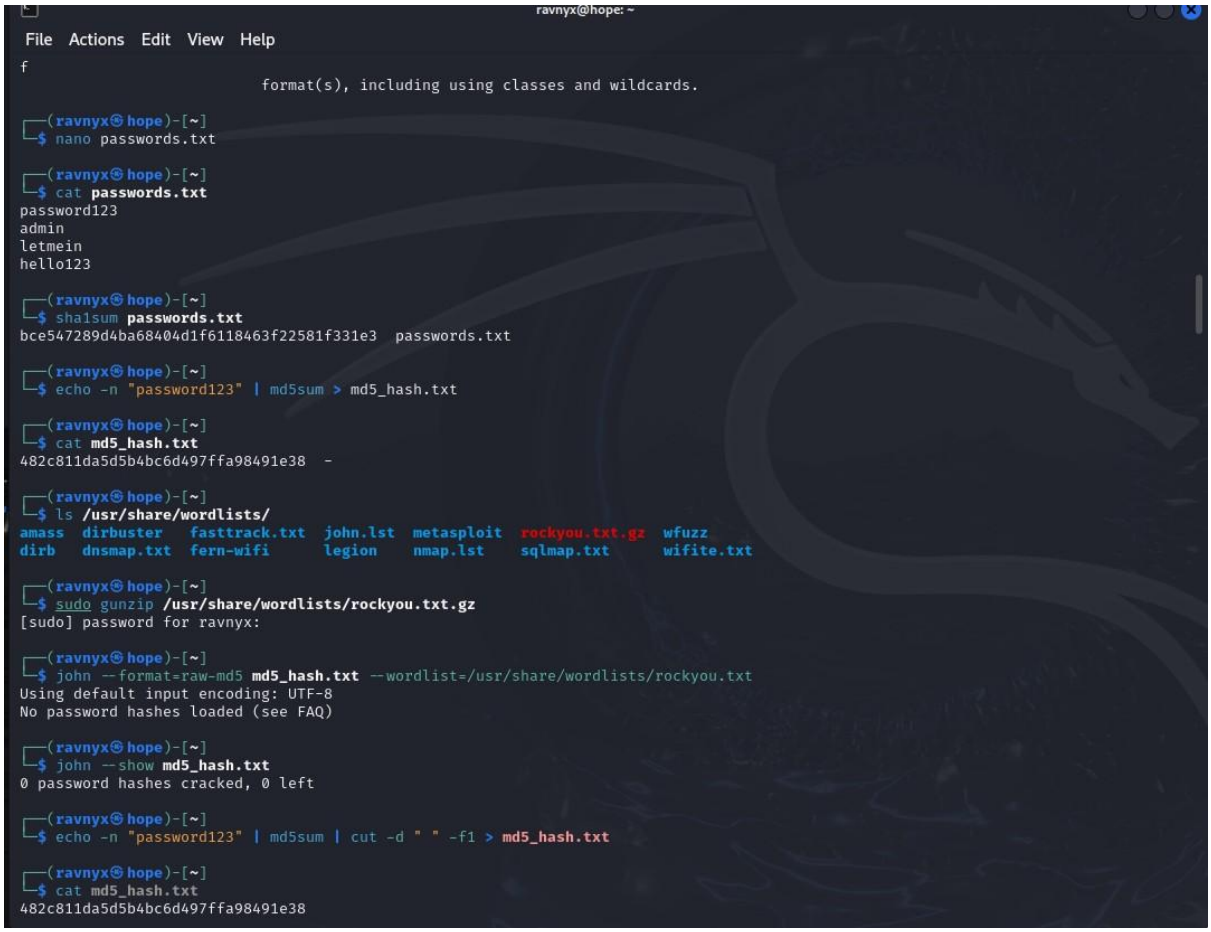
Attack Type	**Approach**	**Speed**	**Effectiveness**
Dictionary Attack	Uses common passwords	Fast	High for weak passwords
Brute-Force Attack	Tries all combinations	Slow	High for short passwords

4.4 Practical Observation from This Task

During the practical experiments, dictionary attacks were successfully used to crack MD5, SHA-256, and even bcrypt hashes when weak passwords were chosen. This demonstrates that the primary weakness often lies in password selection rather than the hashing algorithm alone.

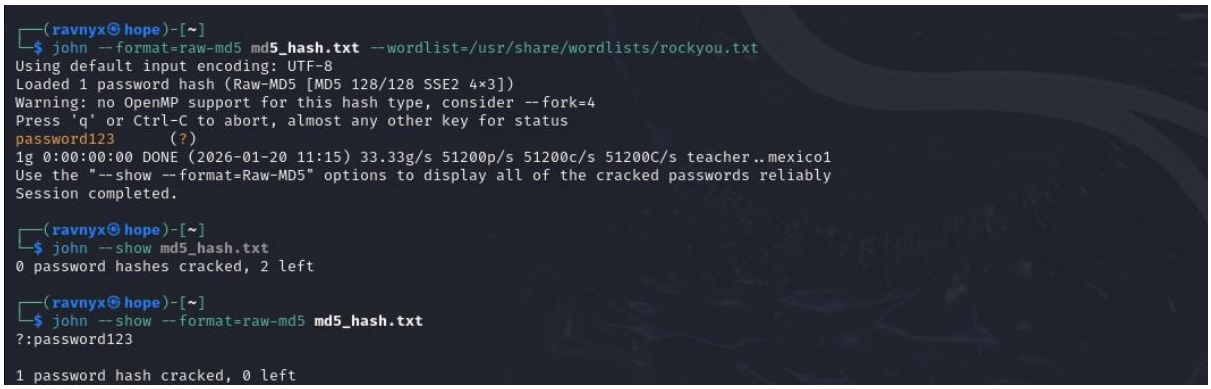
Brute-force attacks were not performed in this task due to their time-intensive nature. However, their theoretical impact highlights the importance of using long, complex passwords and secure hashing algorithms.

Figure 4.1: MD5 Hash Cracking Using John the Ripper



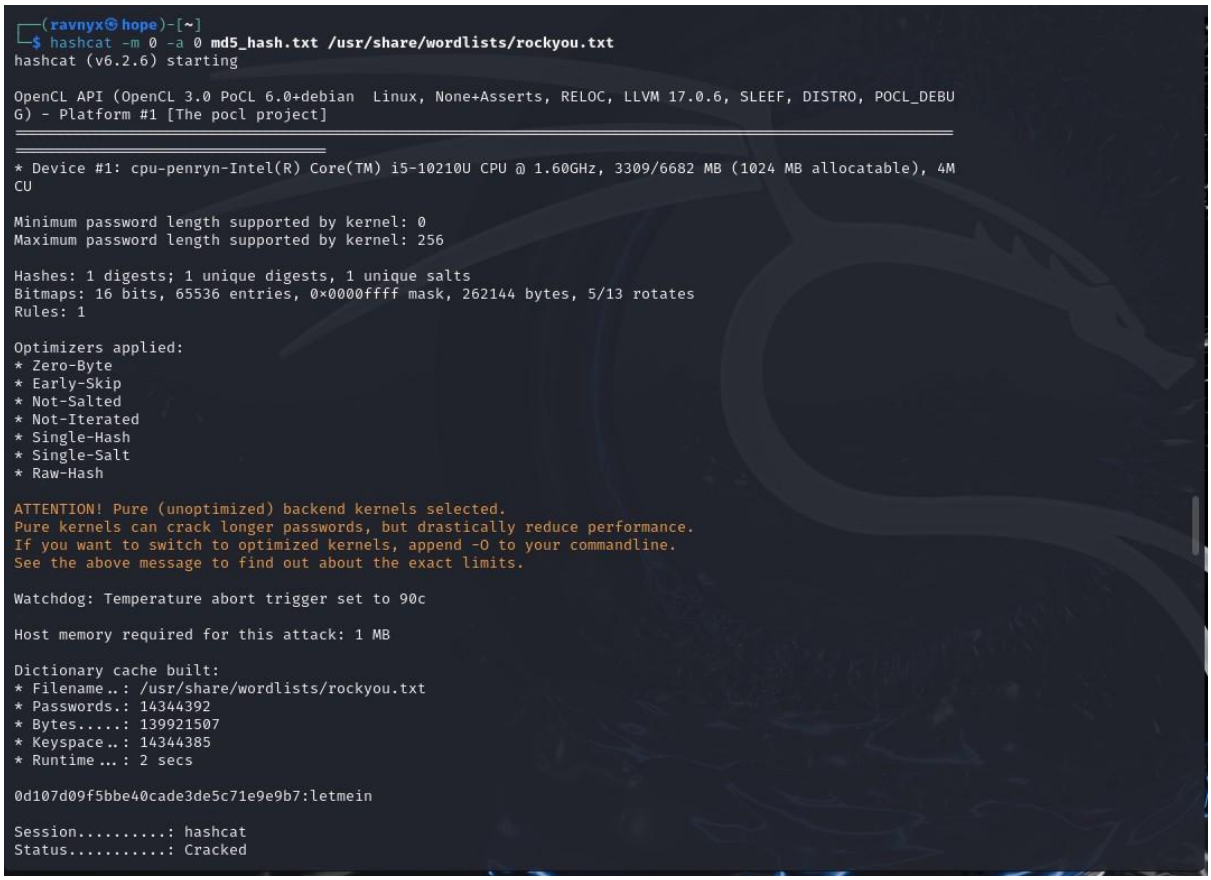
Note: John the Ripper performs a dictionary attack on an MD5 hash using the rockyou.txt wordlist. The rapid recovery of the password highlights the weakness of MD5 when used with common passwords.

Figure 4.2: MD5 Crack Result Verification



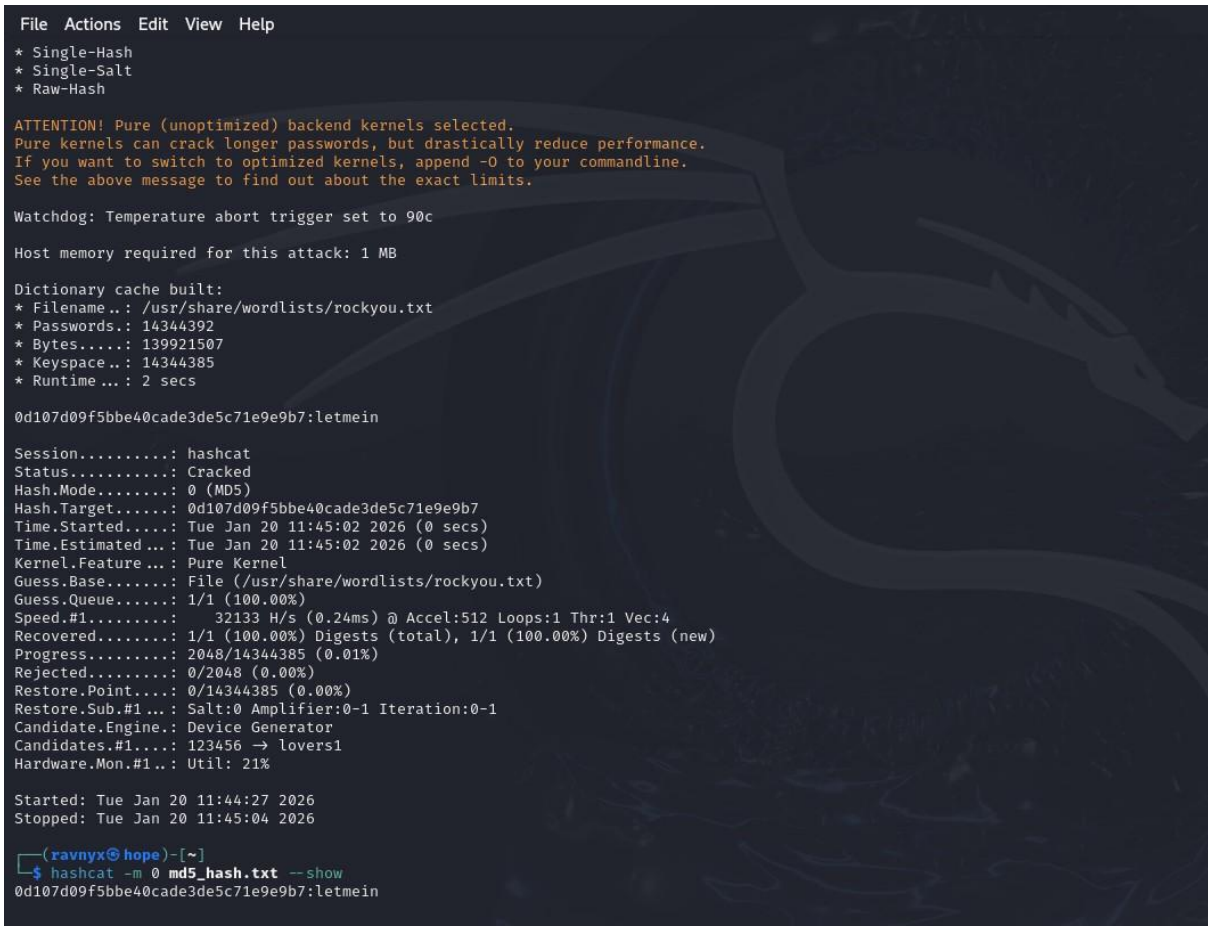
Note: This output confirms the successful recovery of the plaintext password from the MD5 hash, validating the effectiveness of dictionary attacks against weak hashing algorithms.

Figure 4.3: MD5 Hash Cracking using Hashcat



Note: Hashcat demonstrates faster MD5 cracking using optimized CPU/GPU processing. This emphasizes how fast hashing algorithms increase the risk of large-scale password attacks.

Figure 4.4: MD5 Crack Result Using Hashcat

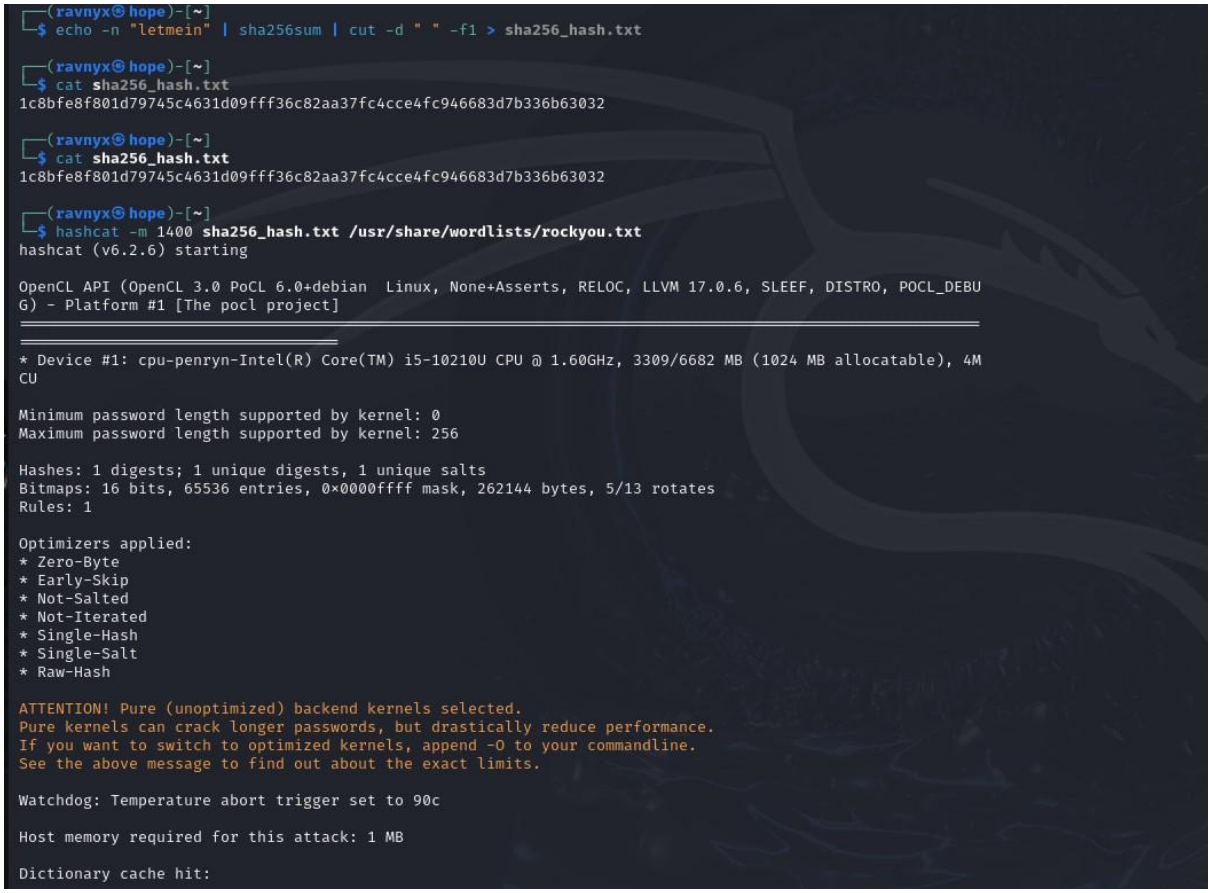


Note: The hash and corresponding plaintext password are displayed in a clear hash:password format, confirming successful password recovery using Hashcat.

Figure 4.5-4.6: SHA-256 Hash Cracking Using John the Ripper and Result Verification

Note: This screenshot shows that although SHA-256 is stronger than MD5, weak passwords can still be cracked using dictionary attacks when fast hashing algorithms are used. The successful recovery of the password demonstrates that stronger hash length alone does not guarantee security if weak passwords are chosen.

Figure 4.7: SHA-256 Hash Cracking Using Hashcat and Result Verification



Note: Hashcat successfully cracks the SHA-256 hash using a dictionary attack, further reinforcing the importance of password strength in addition to hashing algorithms**.**

Chapter 5: Why bcrypt Is Stronger for Password Security

bcrypt is a password hashing algorithm specifically designed to address the weaknesses of older hashing algorithms such as MD5 and SHA-based functions. Unlike general-purpose hashing algorithms, bcrypt was built with password security as its primary objective.

One of the key reasons bcrypt is considered stronger is its **computational cost**. bcrypt is intentionally slow, meaning each hash computation requires a significant amount of processing time. This design makes large-scale password cracking attacks impractical, as attackers cannot test millions of password guesses per second.

Another important feature of bcrypt is its **built-in salting mechanism**. A salt is a random value added to a password before hashing. This ensures that even if two users choose the same password, their stored hashes will be different. Salting also protects against precomputed attacks such as rainbow tables, which are commonly effective against unsalted hashes like MD5 and SHA-1.

bcrypt also supports a **configurable cost factor**, which determines how many iterations are used during hashing. As computing power increases over time, this cost factor can be adjusted to maintain security without changing the underlying algorithm. This makes bcrypt adaptable to future security requirements.

In the practical analysis conducted in this task, bcrypt hashes required significantly more time to crack compared to MD5 and SHA-256 hashes. Even though weak passwords may still be cracked using dictionary attacks, bcrypt greatly increases the effort required by an attacker, reducing the feasibility of large-scale attacks.

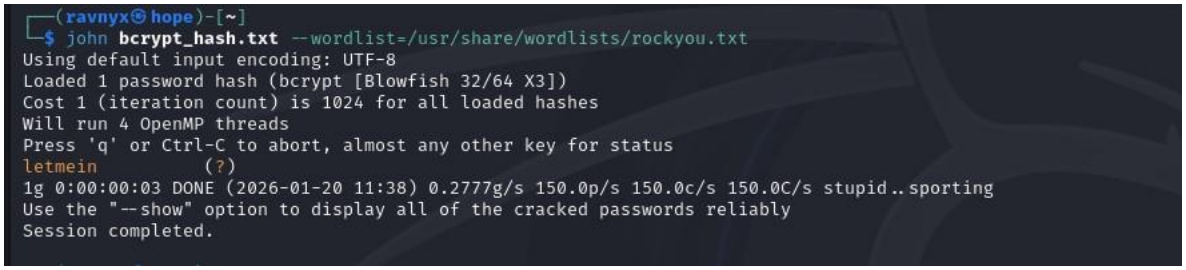
Key Advantages of bcrypt

- Intentionally slow hashing process
- Automatic salting of passwords
- Adjustable cost factor for long-term security
- Strong resistance to brute-force and dictionary attacks

Figure 5.1: bcrypt Hash Generation

Note: This screenshot shows the generation of a bcrypt hash with a defined cost factor. bcrypt automatically applies salting and iterative hashing to enhance password security.

Figure 5.2: bcrypt Cracking Attempt



Note: The bcrypt cracking attempt demonstrates significantly slower performance compared to MD5 and SHA-256, highlighting bcrypt's resistance to high-speed password attacks.

Figure 5.3: bcrypt Crack Result Verification

Note: Although the weak password was eventually recovered, the increased computational effort required shows why bcrypt is recommended for secure password storage.

Chapter 6: Multi-Factor Authentication (MFA) and Best Practices

While strong hashing algorithms such as bcrypt significantly improve password security, passwords alone are no longer sufficient to protect modern systems. If a password is stolen, guessed, or cracked, attackers can still gain unauthorized access. This is where **Multi-Factor Authentication (MFA)** becomes essential.

Multi-Factor Authentication adds an extra layer of security by requiring users to provide more than one form of verification during login. These factors are typically categorized as:

- **Something you know** (password or PIN)
- **Something you have** (OTP, mobile device, security token)
- **Something you are** (biometrics such as fingerprint or facial recognition)

By combining multiple factors, MFA ensures that even if a password is compromised, attackers cannot access the system without the additional authentication factor.

In the context of this task, even though weak passwords were successfully cracked during practical experiments, MFA would have prevented unauthorized access. This highlights the importance of using MFA alongside secure password hashing rather than relying on passwords alone.

Best Practices for Strong Authentication

- Use secure password hashing algorithms such as **bcrypt, scrypt, or Argon2**
- Enforce strong password policies (length, complexity, uniqueness)
- Avoid commonly used or leaked passwords
- Implement **Multi-Factor Authentication (MFA)** wherever possible
- Limit login attempts and apply rate limiting
- Regularly update and audit authentication mechanisms

Chapter 7: Final Observations and Conclusion

While working on this task, I understood how password security works beyond theory. Instead of just reading about hashing algorithms, performing the practical experiments made it clear why some passwords fail easily and why others are harder to break. From the experiments, it was observed that weak and commonly used passwords can be cracked very quickly when fast hashing algorithms like MD5 and SHA-256 are used. Even though SHA-256 is stronger than MD5, it still does not provide enough protection if the password itself is weak. This showed that password strength plays a very important role, regardless of the hashing algorithm. As observed in Figures 4.1–4.7, fast hashing algorithms allow rapid password cracking, whereas bcrypt significantly increases attack cost (Figures 5.1–5.3). When bcrypt was used, the difference was noticeable. The cracking process was much slower due to salting and the higher computational cost. This helped me understand why bcrypt is preferred in real systems, as it makes password cracking more difficult and time-consuming for attackers. Overall, this task helped me understand that password security cannot rely on a single factor. Strong hashing algorithms, good password practices, and additional protections like Multi-Factor Authentication must be used together to properly secure user accounts.